FODS ASSIGNMENT 8

Name – Atharva Shivaji Dabkekar
Roll no. – 2090138
PRN - 1251090357
Division – S.Y A
Batch – 2
DATE – 01-08-2026 TIME – 8:00 TO 10:00

Practical 8: Singly Linked List

Implement a Singly Linked List using dynamic memory allocation. Perform insertion, deletion, traversal, searching, sorting, and concatenation operations.

Answer:

A. Insertion:

1. Algorithm: insert_end()

Purpose: Insert a new node at the end of the linked list.

1. Start.
2. Set H = H0.
3. Traverse the list until H->next == NULL.
4. Create a new node newnode.
5. Store value in newnode->data.
6. Set newnode->next = NULL.
7. Set H->next = newnode.
8. Stop.

Code:

```
Node * H = H0;
  while(H->next != NULL){
    H = H->next;
  }
  Node * newnode = (struct Node*)malloc(sizeof(Node));
  newnode->data = value;
  newnode->next = nullptr;
```

H->next = newnode;

2. Algorithm: insert_begin()

Purpose**:** Insert a new node at the beginning of the linked list.

1. Start.
2. Create a new node newnode.
3. Store value in newnode->data.
4. Set newnode->next = H0.
5. Update H0 = newnode.
6. Stop.

Code:

```
Node * newnode = (struct Node*)malloc(sizeof(Node));
newnode->data = value;
newnode->next = H0;
H0 = newnode;
```

3. Algorithm: insert_at()

Purpose: Insert a new node at a specific position.

1. Start.
2. Set pre = H0.
3. Traverse the list until the node just before the required position is reached.
4. Create a new node newnode.
5. Store value in newnode->data.
6. Set newnode->next = pre->next.
7. Set pre->next = newnode.
8. Stop.

Code: `Node * pre = H0;`

```
for(int i=1;i<pos-1;i++){
   pre = pre->next;
}
Node * newnode = (struct Node*)malloc(sizeof(Node));
newnode->data = value;
newnode->next = pre->next;
pre->next = newnode;
```

```
PS C:\Users\agamb\OneDrive\Desktop\agam.cpp> cd "c:\Users\agamb\OneD
enter the value :2
enter the value :3
enter the value :4
enter the value :5
enter the value :6

 LINKED LIST MENU :
1. Insert at End
2. Insert at Beginning
3. Insert at Specific Position
4. Exit
Enter your choice: 1
Enter the element to insert at end: 7
Insertion at end completed.

 LINKED LIST MENU :
1. Insert at End
2. Insert at Beginning
3. Insert at Specific Position
4. Exit
Enter your choice: 2
Enter the element to insert at beginning: 1
Insertion at beginning completed.

 LINKED LIST MENU :
1. Insert at End
2. Insert at Beginning
3. Insert at Specific Position
4. Exit
Enter your choice: 3
Enter the position to insert: 3
Enter the element to insert at 3: -1
Insertion at position 3 completed.

 LINKED LIST MENU :
1. Insert at End
2. Insert at Beginning
3. Insert at Specific Position
4. Exit
Enter your choice: 4
Exiting switchcase
ur linked list = 1 2 -1 3 4 5 6 7
PS C:\Users\agamb\OneDrive\Desktop\agam.cpp\c++\fods_Assignments>
```

B. Deletion:

1. Algorithm: Deletion at Beginning

1. Start.
2. If H0 == NULL, display "Linked List is Empty" and Stop.
3. Set TEMP = H0.
4. Set H0 = H0 → NEXT.
5. Delete TEMP.
6. Stop.

2. Algorithm: Deletion at End

1. Start.
2. If H0 == NULL, display "Linked List is Empty" and Stop.
3. If H0 → NEXT == NULL, delete H0 and Stop.
4. Set PRE = H0.
5. Repeat until PRE → NEXT → NEXT == NULL:
    - Set PRE = PRE → NEXT.
6. Set TEMP = PRE → NEXT.
7. Set PRE → NEXT = NULL.
8. Delete TEMP.
9. Stop.

3. Algorithm: Deletion at Specific Position

1. Start.
2. If H0 == NULL, display "Linked List is Empty" and Stop.
3. If POS == 1, delete the first node and Stop.
4. Set PRE = H0.
5. Repeat for i = 1 to POS - 2:
    - Set PRE = PRE → NEXT.
6. If PRE → NEXT == NULL, display "Invalid Position" and Stop.
7. Set TEMP = PRE → NEXT.

8. Set PRE → NEXT = TEMP → NEXT.
9. Delete TEMP.
10. Stop.

```
PS C:\Users\agamb\OneDrive\Desktop\agam.cpp> cd "c:\Users\agamb\One
enter the value :1
enter the value :2
enter the value :3
enter the value :4
enter the value :5
enter the value :6
enter the value :7
enter the value :8

 LINKED LIST MENU :
1. delete at End
2. delete at Beginning
3. delete at Specific Position
4. Exit
Enter your choice: 1
Deletion at beginning completed.

 LINKED LIST MENU :
1. delete at End
2. delete at Beginning
3. delete at Specific Position
4. Exit
Enter your choice: 2
Deletion at end completed.

 LINKED LIST MENU :
1. delete at End
2. delete at Beginning
3. delete at Specific Position
4. Exit
Enter your choice: 3
Enter the position to delete: 4
Deletion at position 4 completed.

 LINKED LIST MENU :
1. delete at End
2. delete at Beginning
3. delete at Specific Position
4. Exit
Enter your choice: 4
Exiting switchcase
ur linked list = 2 3 4 6 7
PS C:\Users\agamb\OneDrive\Desktop\agam.cpp\c++\fods Assignments>
```

C. Treversal:

Algorithm: Traversing a Linked List

1. Start.
2. Set TEMP = H0.
3. Repeat while TEMP != NULL:
    - Display TEMP → DATA.
    - Set TEMP = TEMP → NEXT.
4. Stop.

Code:

```
void display(Node *head){
  cout<<"ur linked list = ";
  while (head != NULL){
    cout<<head->data<<" ";
    head = head->next;
  }
}
```

```
PS C:\Users\agamb\OneDrive\Desktop\agam.cpp> cd "c:\Users\agamb\OneD
enter the size of linked list :5
enter the value :1
enter the value :2
enter the value :3
enter the value :4
enter the value :5
traversing the linked list:
ur linked list = 1 2 3 4 5
PS C:\Users\agamb\OneDrive\Desktop\agam.cpp\c++\fods_Assignments>
```

D. Searching:

Algorithm

1. Start.
2. Set TEMP = HEAD.
3. Read the element to be searched as KEY.
4. Set POS = 1.
5. Repeat while TEMP != NULL:
    - If TEMP → DATA == KEY, display **"Element found"** and its position, then Stop.
    - Set TEMP = TEMP → NEXT.
    - Increment POS by 1.

6. If TEMP == NULL, display **"Element not found"**.
7. Stop.

Code:

```
void linear_search(Node *head, int key) {

  Node *temp = head;
  int pos = 1;

  while (temp != NULL) {

    if (temp->data == key) {
      cout << "Element found at position "
          << pos << endl;
      return;
    }

    temp = temp->next;
    pos++;
  }

  cout << "Element not found" << endl;
}
```

```
PS C:\Users\agamb\OneDrive\Desktop\agam.cpp> cd "c:\Users\agamb\OneDr
enter the size of linked list :6
enter the value :1
enter the value :2
enter the value :3
enter the value :4
enter the value :5
enter the value :6
ur linked list = 1 2 3 4 5 6
enter the value to search :5
Element found at position 5
PS C:\Users\agamb\OneDrive\Desktop\agam.cpp\c++\fods_Assignments>
```

E. Sorting:

Algorithm

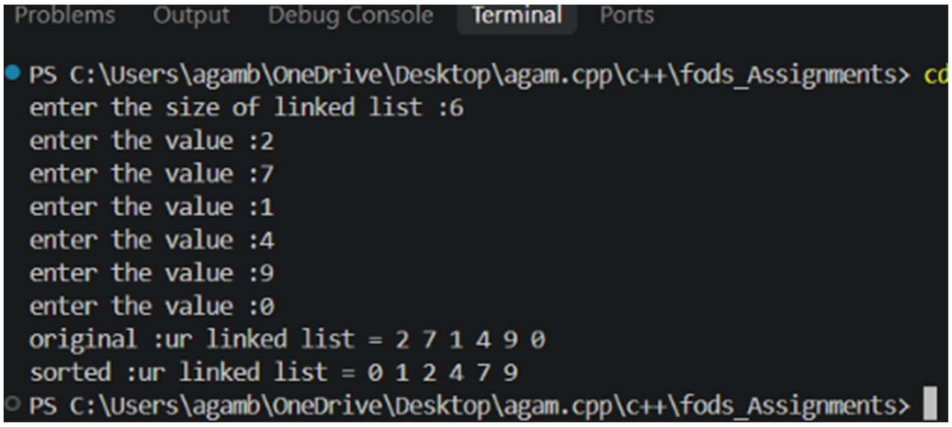
1. Start.
2. If HEAD == NULL, display **"Linked List is Empty"** and Stop.
3. Set I = HEAD.
4. Repeat while I != NULL:
    - Set J = HEAD.
    - Repeat while J → NEXT != NULL:
        - If J → DATA > J → NEXT → DATA:
            - Swap J → DATA and J → NEXT → DATA.
        - Set J = J → NEXT.
    - Set I = I → NEXT.
5.

Code:

```
void bubble_sort(Node *head) {

  if (head == NULL) {
    cout << "Linked List is Empty!" << endl;
    return;
  }
  Node *i;
  Node *j;
  int temp;
  for (i = head; i != NULL; i = i->next) {
    for (j = head; j->next != NULL; j = j->next) {
      if (j->data > j->next->data) {
        temp = j->data;
        j->data = j->next->data;
        j->next->data = temp;
      }
```

```
    }
  }
}
```



F. Concatenation:

Algorithm

1. Start.
2. Create two linked lists HEAD1 and HEAD2.
3. If HEAD1 == NULL, set HEAD1 = HEAD2 and Stop.
4. Set TEMP = HEAD1.
5. Traverse the first linked list until TEMP → NEXT == NULL.
6. Set TEMP → NEXT = HEAD2.
7. The two linked lists are now concatenated.
8. Display the concatenated linked list.
9. Stop.

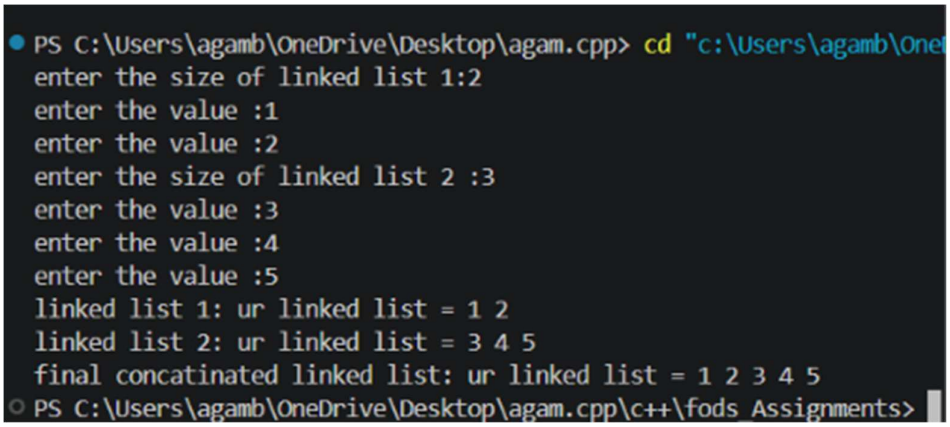
Code:

```
void concatenate(Node *head1, Node *head2) {

  if (head1 == NULL) {
    head1 = head2;
    return;
  }

  Node *temp = head1;
  while (temp->next != NULL) {
    temp = temp->next;
  }

  temp->next = head2;
}
```


PS C:\Users\agamb\OneDrive\Desktop\agam.cpp> cd "c:\Users\agamb\One
enter the size of linked list 1:2
enter the value :1
enter the value :2
enter the size of linked list 2 :3
enter the value :3
enter the value :4
enter the value :5
linked list 1: ur linked list = 1 2
linked list 2: ur linked list = 3 4 5
final concatinated linked list: ur linked list = 1 2 3 4 5
PS C:\Users\agamb\OneDrive\Desktop\agam.cpp\c++\fods Assignments>